

EXPERIMENT - 12

AIM : Implement a Spring Boot and Spring Security based application to demonstrate use of authentication and authorization.

Student.java

```
package com.example.demo.entity;

import jakarta.persistence. Entity;
import jakarta.persistence. GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private String email;

    private int semester;

    private String city;

    private String gender;

    private String program;
```

```

public Student() {
}

public Long getId() {
    return id;
}

public String getName() {
    return name;
}

public String getEmail() {
    return email;
}

public int getSemester() {
    return semester;
}

public String getCity() {
    return city;
}

public String getGender() {
    return gender;
}

public String getProgram() {
    return program;
}

    public Student(Long id, String name, String email, int
semester, String city, String gender, String program) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.semester = semester;
        this.city = city;
    }

```

```

        this.gender = gender;
        this.program = program;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public void setSemester(int semester) {
        this.semester = semester;
    }

    public void setCity(String city) {
        this.city = city;
    }

    public void setGender(String gender) {
        this.gender = gender;
    }

    public void setProgram(String program) {
        this.program = program;
    }
}

```

StudentErrorResponse.java

```

package com.example.demo.dto;
public class StudentErrorResponse {

```

```

private int status;
private String message;
private long timestamp;

public int getStatus() {
    return status;
}

public void setStatus(int status) {
    this.status = status;
}

public String getMessage() {
    return message;
}

public void setMessage(String message) {
    this.message = message;
}

public long getTimestamp() {
    return timestamp;
}

public void setTimestamp(long timestamp) {
    this.timestamp = timestamp;
}
}

```

AppSecurityConfiguration.java

```

package com.example.demo.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.Customizer;

```

```

import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.provisioning.UserDetailsService;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class AppSecurityConfiguration {

    @Bean
    public UserDetailsManager userDetailsManager() {

        UserDetails u1 = User.builder()
            .username("user1")
            .password("{noop}admin123")
            .roles("STUDENT")
            .build();

        UserDetails u2 = User.builder()
            .username("user2")
            .password("{noop}admin1")
            .roles("TEACHER")
            .build();

        UserDetails u3 = User.builder()
            .username("user3")
            .password("{noop}admin")
            .roles("TEACHER", "ADMIN")
            .build();

        return new InMemoryUserDetailsManager(u1, u2, u3);
    }
}

```

```

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.authorizeHttpRequests(configurer -> configurer
            .requestMatchers(HttpMethod.GET, "/api/students")
                .hasAnyRole("STUDENT", "TEACHER", "ADMIN")
                .requestMatchers(HttpMethod.GET, "/api/students/**")
                .hasAnyRole("STUDENT", "TEACHER", "ADMIN")
                .requestMatchers(HttpMethod.POST, "/api/students")
                .hasRole("TEACHER")
                .requestMatchers(HttpMethod.PATCH, "/api/students/**")
                .hasRole("TEACHER")
                .requestMatchers(HttpMethod.DELETE, "/api/students/**")
                .hasRole("ADMIN")
            );
        http.httpBasic(Customizer.withDefaults());
        return http.build();
    }
}

```

StudentController.java

```

package com.example.demo.restcontroller;

import com.example.demo.entity.Student;
import com.example.demo.service.StudentService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

```

```

@RestController
@RequestMapping("/api")
public class StudentController {

    StudentService studentService;

    @Autowired
    StudentController(StudentService studentService) {
        this.studentService = studentService;
    }

    @GetMapping("/students")
    public List<Student> getStudents() {
        return studentService.findAll();
    }

    @GetMapping("/students/{studentId}")
    Student getStudent(@PathVariable Long studentId) {
        return studentService.findById(studentId);
    }

    @PostMapping("/students")
    Student addStudent(@RequestBody Student student) {
        return studentService.save(student);
    }

    @PatchMapping("/students/{studentId}")
    Student updateStudent(@PathVariable Long studentId, @RequestBody Student student) {
        return studentService.update(studentId, student);
    }

    @DeleteMapping("/students/{studentId}")
    String deleteStudent(@PathVariable Long studentId) {
        studentService.deleteById(studentId);
        return "Student with id " + studentId + " has been deleted.";
    }
}

```

```
}  
  
}
```

StudentRepository.java

```
package com.example.demo.repository;  
  
import com.example.demo.entity.Student;  
import org.springframework.data.jpa.repository.JpaRepository;  
  
public interface StudentRepository extends JpaRepository<Student, Long> {  
}
```

StudentExceptionHandler.java

```
package com.example.demo.exception;  
  
import com.example.demo.dto.StudentErrorResponse;  
import org.springframework.http.HttpStatus;  
import org.springframework.http.ResponseEntity;  
import org.springframework.web.bind.annotation.ControllerAdvice;  
import org.springframework.web.bind.annotation.ExceptionHandler;  
  
@ControllerAdvice  
public class StudentExceptionHandler {  
  
    @ExceptionHandler  
    ResponseEntity<StudentErrorResponse> handleException(StudentNotFoundException studentNotFoundException) {  
        StudentErrorResponse errorResponse = new StudentErrorResponse();  
        errorResponse.setStatus(HttpStatus.NOT_FOUND.value());  
    }  
}
```



```

        errorResponse.setMessage(String.valueOf(studentNotFoundException.getMessage()));
        errorResponse.setTimestamp(System.currentTimeMillis());
        return new ResponseEntity<>(errorResponse, HttpStatus.NOT_FOUND);
    }
}

```

StudentNotFoundException.java

```

package com.example.demo.exception;

public class StudentNotFoundException extends RuntimeException{

    public StudentNotFoundException(String message) {
        super(message);
    }
}

```

StudentService.java

```

package com.example.demo.service;
import com.example.demo.entity.Student;
import java.util.List;

public interface StudentService {

    List<Student> findAll();
    Student findById(Long id);
    Student save(Student student);
    Student update(Long studentId, Student student);
    void deleteById(Long id);

}

```

StudentServiceImpl.java

```
package com.example.demo.service;

import com.example.demo.entity.Student;
import com.example.demo.exception.StudentNotFoundException;
import com.example.demo.repository.StudentRepository;
import jakarta.transaction.Transactional;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;

@Service
public class StudentServiceImpl implements StudentService {

    StudentRepository studentRepository;

    @Autowired
    StudentServiceImpl(StudentRepository studentRepository)
    {
        this.studentRepository = studentRepository;
    }

    @Override
    public List<Student> findAll() {
        return studentRepository.findAll();
    }

    @Override
    public Student findById(Long id) {
        Optional<Student> optional = studentRepository.findById(id);
        Student student = null;

        if (optional.isPresent()) {
```

```

        student = optional.get();
    } else {
        throw new StudentNotFoundException("Student not
found for id: " + id);
    }

    return student;
}

@Override
@Transactional
public Student save(Student student) {
    return studentRepository.save(student);
}

@Transactional
@Override
public Student update(Long studentId, Student student)
{
    Student existingStudent = findById(studentId);
    if (existingStudent != null) {
        if (student.getName() != null) {
            existingStudent.setName(student.getName());
        }
        if (student.getEmail() != null) {
            existingStudent.setEmail(student.getEmail
());
        }
        if (student.getSemester() != 0) {
            existingStudent.setSemester(student.getSeme
ster());
        }
        if (student.getCity() != null) {
            existingStudent.setCity(student.getCity());
        }
        if (student.getGender() != null) {
            existingStudent.setGender(student.getGender
());
        }
    }
}

```

```

        }
        if (student.getProgram() != null) {
            existingStudent.setProgram(student.getProgram());
        }
    }
    if(existingStudent == null) {
        throw new StudentNotFoundException("Student not found for id: " + studentId);
    }
    return studentRepository.save(existingStudent);
}

@Transactional
@Override
public void deleteById(Long id) {
    Student student = findById(id);
    studentRepository.delete(student);
}
}

```

OUTPUT

After adding basic security

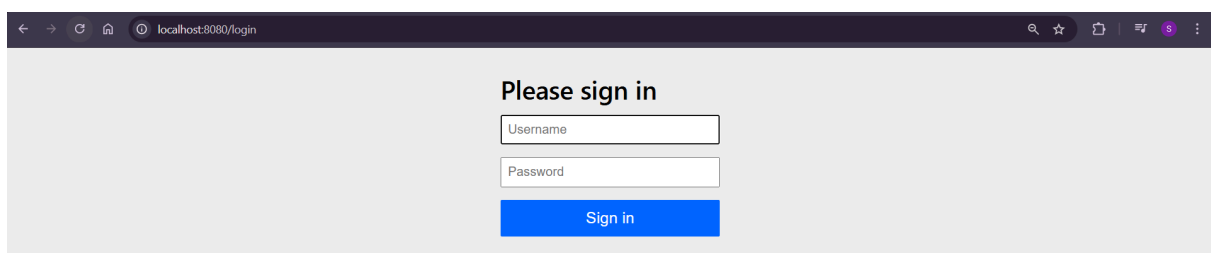
```

Using generated security password: 2ef8cbc2-9dd9-4a42-a8e3-dbf564fbf16

This generated password is for development use only. Your security configuration must be updated before running your application in production.

2026-03-04T22:46:38.821+05:30 INFO 27340 --- [lab11] [main] r$InitializeUserDetailsManagerConfigurer : Global AuthenticationManager configured with UserDetails
2026-03-04T22:46:39.064+05:30 INFO 27340 --- [lab11] [main] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2026-03-04T22:46:39.072+05:30 INFO 27340 --- [lab11] [main] com.example.demo.Lab11Application : Started Lab11Application in 5.625 seconds (process running for 1.001)

```



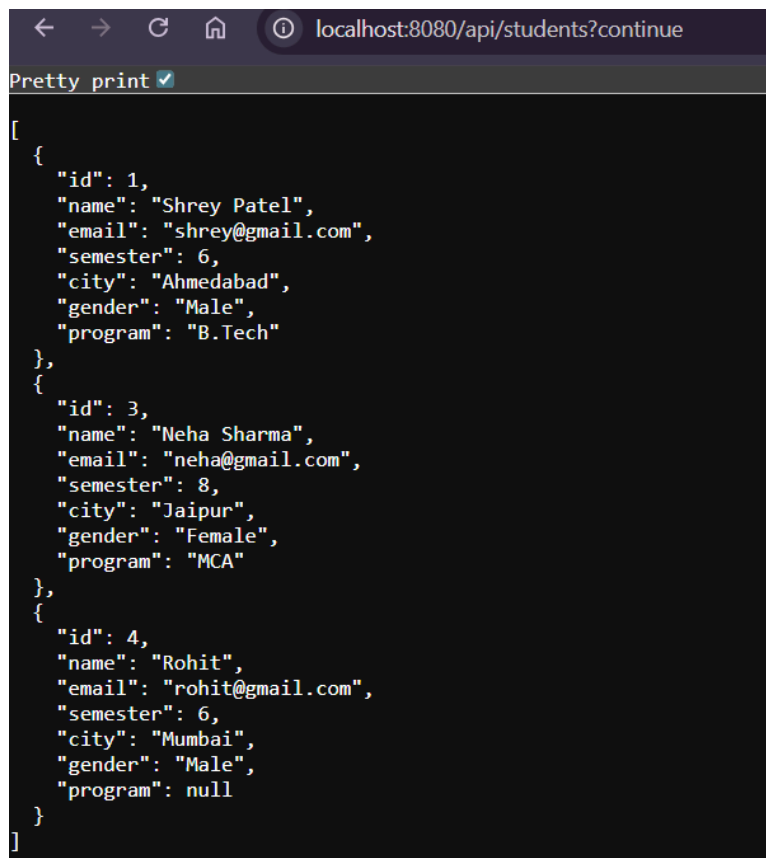
localhost:8080/login

Please sign in

Username

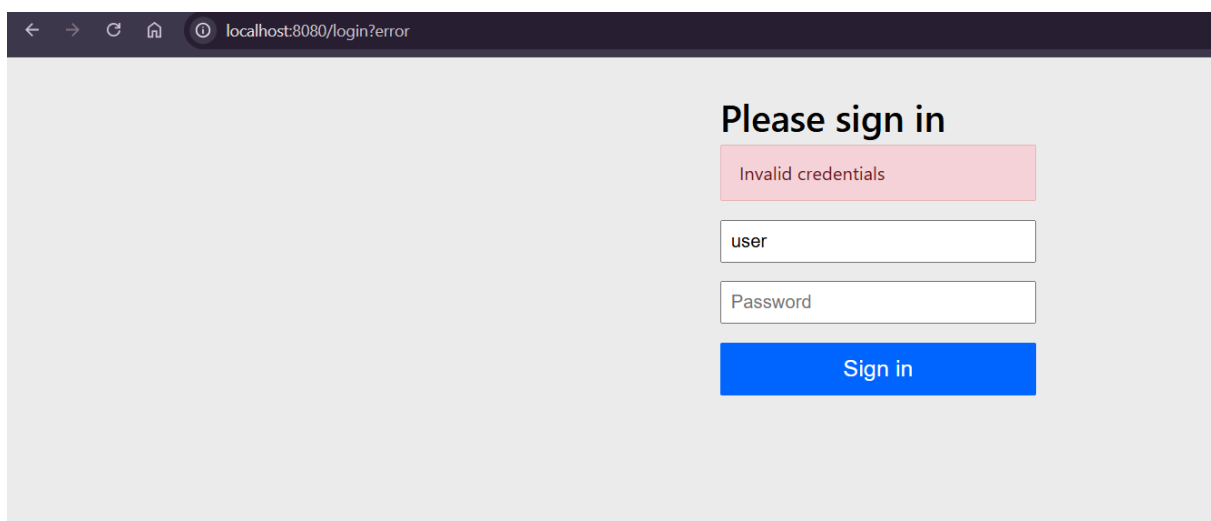
Password

Sign in



A screenshot of a web browser window with the address bar showing `localhost:8080/api/students?continue`. The page content displays a JSON array of three student objects, formatted with "Pretty print" checked. The data is as follows:

```
[
  {
    "id": 1,
    "name": "Shrey Patel",
    "email": "shrey@gmail.com",
    "semester": 6,
    "city": "Ahmedabad",
    "gender": "Male",
    "program": "B.Tech"
  },
  {
    "id": 3,
    "name": "Neha Sharma",
    "email": "neha@gmail.com",
    "semester": 8,
    "city": "Jaipur",
    "gender": "Female",
    "program": "MCA"
  },
  {
    "id": 4,
    "name": "Rohit",
    "email": "rohit@gmail.com",
    "semester": 6,
    "city": "Mumbai",
    "gender": "Male",
    "program": null
  }
]
```



A screenshot of a web browser window with the address bar showing `localhost:8080/login?error`. The page has a light gray background and contains a sign-in form on the right side. The form includes a title, an error message, input fields for user and password, and a sign-in button.

Please sign in

Invalid credentials

user

Password

Sign in

After adding authorization

My Collection / New

GET http://localhost:8080/api/students Send

Docs Params **Authorization** Headers (10) Body Scripts Settings Cookies

Auth Type: Basic Auth

The authorization header will be automatically generated when you send the request. Learn more about [Basic Auth](#) authorization.

Username: user1

Password:

Body Cookies (1) Headers (11) Test Results 200 OK • 293 ms • 686 B Save Response

JSON Preview Visualize

```
1 [
2   {
3     "id": 1,
4     "name": "Shrey Patel",
5     "email": "shrey@gmail.com",
6     "semester": 6,
7     "city": "Ahmedabad",
8     "gender": "Male",
9     "program": "B.Tech"
10  },
11  ]
```

GET http://localhost:8080/api/students Send

Docs Params **Authorization** Headers (10) Body Scripts Settings Cookies

Auth Type: Basic Auth

The authorization header will be automatically generated when you send the request. Learn more about [Basic Auth](#) authorization.

Username: user3

Password: admin

Body Cookies (1) Headers (11) Test Results 200 OK • 254 ms • 686 B Save Response

JSON Preview Visualize

```
1 [
2   {
3     "id": 1,
4     "name": "Shrey Patel",
5     "email": "shrey@gmail.com",
6     "semester": 6,
7     "city": "Ahmedabad",
8     "gender": "Male",
9     "program": "B.Tech"
10  },
11  ]
```

DELETE

http://localhost:8080/api/students/1

Send

DocsParamsAuthorizationHeaders (10)BodyScriptsSettings

Cookies

Auth Type

Basic Auth

The authorization header will be automatically generated when you send the request. Learn more about **Basic Auth** authorization.

Username

user2

Password

admin1

BodyCookies (1)Headers (12)Test Results

401 Unauthorized · 122 ms · 417 B · Save Response

Raw

Preview

Pass the correct auth credentials

1